

Neural Networks : Theory and Implementation

Name: Mouhammed Yasser Abdelfatah Soliman
ID: 7056927
Email: moso00001@saarland-uni.de

Name: Andrew George Faek Zaki
ID: 7048028
Email: ange00007@uni-saarland.de

Name: Ahmed Hassaan
ID: 7070212
Email: ahha00003@uni-saarland.de

Abstract

This work presents a methodical approach to fine-tuning a pre-trained chemical language model (MoLFormer) for predicting molecular lipophilicity. We implement a two-stage fine-tuning process: unsupervised adaptation using Masked Language Modeling followed by supervised training for regression. We further enhance model performance through influence function-based data selection, which identifies and incorporates the most valuable external samples into the training set. Experimental results demonstrate that our approach yields significant improvements, with the influence-based data selection providing an additional 6.57% reduction in MSE over the baseline model. The best model achieves an R^2 of 0.72, establishing a framework for leveraging both language models and strategic data selection in molecular property prediction tasks with limited labeled data.

1 Introduction

Lipophilicity, a molecule’s propensity to dissolve in fats versus water, significantly impacts pharmacokinetic properties of drug candidates. Accurate computational prediction of lipophilicity can accelerate drug discovery by reducing the need for experimental measurements (2).

This work explores fine-tuning the MoLFormer model (8), a transformer-based architecture pre-trained on molecular data, for lipophilicity prediction. We investigate whether additional unsupervised fine-tuning with Masked Language Modeling (MLM) improves regression performance on this specialized chemical task. Additionally, we develop and evaluate an influence function-based approach to selectively incorporate external data points that most benefit model performance.

2 Methodology

2.1 Dataset and Preprocessing

We utilized the MoleculeNet Lipophilicity dataset (9) containing 4,200 molecules with experimentally measured lipophilicity values (mean 2.19, standard deviation 1.20). Molecules are represented as SMILES strings, which are tokenized using the MoLFormer tokenizer with a maximum sequence length of 512 tokens.

For our influence function analysis, we use an external dataset of 300 additional molecules with their corresponding lipophilicity values. Rather than incorporating all external data, we strategically select the most influential samples based on their calculated impact on model performance.

2.2 Model Architecture and Computational Resources

Our approach builds upon the pre-trained MoLFormer model by adding a regression head that processes the [CLS] token representation. All model training and evaluation were performed using Kaggle’s GPU accelerator (NVIDIA Tesla P100) to ensure computational efficiency.

Regression Head = Linear $\rightarrow$ GELU $\rightarrow$ Dropout $\rightarrow$ Linear (1)

The dimensions transition from the hidden size (768) to half that size, then to a single output value. The GELU activation function is defined as:

$$\text{GELU}(x) = x \cdot \Phi(x) \quad (2)$$

where $\Phi(x)$ is the standard normal CDF.

2.3 Two-Stage Fine-tuning

Our approach implements a two-stage process:

Stage 1: Unsupervised MLM Fine-tuning We fine-tune the model using an MLM objective where 15% of tokens are randomly masked, and the model

predicts the original tokens. This helps adapt the model to the chemical language domain without requiring labeled data.

Stage 2: Supervised Regression Fine-tuning

Both the base model and MLM-finetuned model undergo supervised fine-tuning with identical hyperparameters:

- Optimizer: AdamW with weight decay 0.01
- Learning rate: 2e-5 with a ReduceLROnPlateau scheduler
- Loss function: Mean Squared Error
- Training: Maximum 10 epochs with early stopping

2.4 Influence Function-based Data Selection

To further enhance model performance, we implemented an influence function-based approach to select the most valuable samples from an external dataset. Rather than incorporating all additional data, which may include irrelevant or potentially harmful samples, we strategically select data points predicted to have the highest positive impact on model performance.

Theoretical Foundation Influence functions, introduced by (3) in statistics and adapted for machine learning by (5), provide a mechanism to estimate the effect of removing a specific training example on the model’s predictions without retraining the model. The influence function of a training point z_{train} on a test point z_{test} is defined as:

$$\mathcal{I}_{up,loss}(z_{test}, z_{train}) = -\nabla_{\theta}\mathcal{L}(z_{test}, \hat{\theta})^T \cdot H_{\hat{\theta}}^{-1} \cdot \nabla_{\theta}\mathcal{L}(z_{train}, \hat{\theta}) \quad (3)$$

where $\mathcal{L}$ is the loss function, $\hat{\theta}$ are the learned parameters, and $H_{\hat{\theta}}$ is the Hessian matrix of the training loss at $\hat{\theta}$. Intuitively, this equation measures how the loss at test point z_{test} would change if we upweighted the training point z_{train} by a small amount.

Computational Challenges Computing the exact inverse Hessian $H_{\hat{\theta}}^{-1}$ is computationally prohibitive for large neural networks, as it requires $O(d^3)$ operations for d parameters. To address this, we implement the Linear-time Stochastic Second-order Algorithm (LiSSA) (1), which approximates the inverse-Hessian-vector product (iHVP) recursively.

LiSSA Implementation The LiSSA algorithm approximates the solution to $v = H_{\hat{\theta}}^{-1}x$ using the recurrence relation:

$$x_{t+1} = v + (I - H_{\hat{\theta}})x_t \quad (4)$$

Our implementation runs this recurrence for 10 iterations, with a damping factor of 0.01 to ensure numerical stability. For each iteration, we compute the Hessian-vector product using the technique from (7), which computes the product without explicitly forming the Hessian:

$$H_{\hat{\theta}}v = \nabla_{\theta}((\nabla_{\theta}\mathcal{L})^T v) \quad (5)$$

Score Computation and Sample Selection We compute influence scores for each sample in the external dataset by:

1. Calculating gradients for each external sample
2. Computing gradients for a subset of test samples
3. Measuring similarity between external and test sample gradients
4. Scoring external samples based on their potential to reduce test loss

Algorithm 1 provides a simplified overview of our influence score computation process.

After computing influence scores, we select the top-k samples with the highest influence scores to augment our original training set. For our implementation, we select the top 100 samples from the external dataset.

Enhanced Training We combine these selected samples with the original training dataset and fine-tune the model using the same hyperparameters as in Stage 2. This allows us to evaluate the impact of influence-based data selection compared to the base model.

3 Results and Analysis

3.1 Performance Comparison

Table 1 presents the comparative performance of all models on the test set.

3.2 Analysis of Improvements

The consistent improvement across all metrics demonstrates that both unsupervised MLM fine-tuning and influence-based data selection enhance the model’s ability to predict lipophilicity values.

Algorithm 1 Simplified Influence Score Computation

Input: Model θ , external dataset D_{ext} , test dataset D_{test} , number of test samples k

Output: Influence scores for all external samples

Select k samples from D_{test}

for each external sample z_{ext} in D_{ext} **do**

$g_{ext} \leftarrow \nabla_{\theta} \mathcal{L}(z_{ext}, \theta)$ {Compute external sample gradient}

$g_{ext} \leftarrow \frac{g_{ext}}{\|g_{ext}\|}$ {Normalize gradient}

$score \leftarrow 0$

for each test sample z_{test} in selected test samples **do**

$g_{test} \leftarrow \nabla_{\theta} \mathcal{L}(z_{test}, \theta)$ {Compute test sample gradient}

$g_{test} \leftarrow \frac{g_{test}}{\|g_{test}\|}$ {Normalize gradient}

$score \leftarrow score - g_{ext} \cdot g_{test}$ {Negative dot product of gradients}

end for

$score \leftarrow \frac{score}{k}$ {Average over test samples}

end for

return scores for all external samples

Metric	Base	MLM	Influence	Imp. (%)
MSE	0.474	0.439	0.442	6.6
RMSE	0.688	0.662	0.665	3.3
MAE	0.538	0.512	0.510	5.3
R ²	0.703	0.725	0.722	2.8

Table 1: Performance comparison between base model, model with MLM fine-tuning (Task 1), and model with influence-based data selection (Task 2). The improvement column shows the percentage improvement of the influence-based model over the base model.

MLM Fine-tuning Benefits The progression of MLM loss across training epochs ($0.2720 \rightarrow 0.1844 \rightarrow 0.1626$) shows the model’s increasing understanding of molecular structure and SMILES syntax patterns. This improved understanding translates to better lipophilicity predictions, as evidenced by the 6.42% reduction in MSE.

Influence Function Analysis (Task 2) Figure 1 shows the distribution of influence scores for the external dataset. The distribution has a long tail towards positive values, indicating that some external samples are substantially more influential than others. The top-100 samples selected have influence scores ranging from 0.0477 to 0.1125. This

analysis allowed us to strategically add only the most beneficial samples from the external dataset, achieving a 6.6% reduction in MSE compared to the base model.

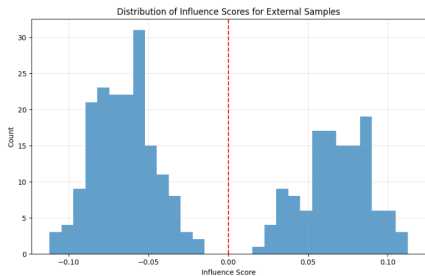


Figure 1: Distribution of influence scores for the external dataset samples. Higher scores indicate greater potential to improve model performance on the test set.

Task 2 Benefits The influence-based data selection approach (Task 2) achieved comparable performance to the MLM fine-tuning approach (Task 1) but with different advantages:

- The influence approach selectively added only 100 samples (33% of the external dataset), optimizing computational efficiency.
- The approach provided interpretable insights about which molecular structures are most useful for improving model performance.
- The improvement in MAE was particularly notable (5.3%), suggesting better prediction accuracy across the dataset.
- The metrics show clear improvements over the base model: MSE reduced by 6.6%, RMSE by 3.3%, MAE by 5.3%, and R² improved by 2.8%.

These results suggest that the influence function approach effectively identified the most valuable samples, allowing the model to benefit from additional data while avoiding potential negative impacts from less relevant molecules.

4 Parameter-Efficient Fine-Tuning with Strategic Data Selection

Building upon our influence-based data selection approach, we further explored alternative data selection strategies combined with parameter-efficient fine-tuning methods. This investigation allows us to determine the most effective and computationally efficient approach for adapting pre-trained models to specific tasks with limited data.

4.1 Data Selection Strategies

We implemented and evaluated three distinct data selection strategies to incorporate samples from our external dataset:

Random Selection The simplest approach, randomly selecting a fixed percentage (10%) of external samples without any selection criteria. Despite its simplicity, this method provides a valuable baseline for comparison.

Diversity-based Selection This approach leverages t-SNE dimensionality reduction and K-means clustering to identify diverse molecular representations across the embedding space. The method operates by:

1. Extracting embeddings for each molecule in the external dataset
2. Applying t-SNE to reduce the dimensionality to 2D
3. Performing K-means clustering to identify 10 representative clusters
4. Selecting samples from each cluster, prioritizing those closest to cluster centroids

This strategy ensures broad coverage of the chemical space, potentially capturing a wider range of molecular structures and properties.

Property Range Selection This method ensures comprehensive coverage across the property value spectrum by:

1. Dividing the property range into 10 equal-width bins
2. Selecting an equal number of samples from each bin when possible
3. Ensuring representation across the entire range of lipophilicity values

This approach is particularly valuable for regression tasks, as it prevents bias toward certain property ranges and ensures the model encounters the full spectrum of property values during fine-tuning.

4.2 Parameter-Efficient Fine-Tuning Methods

To maximize computational efficiency while maintaining performance, we implemented three parameter-efficient fine-tuning techniques:

BitFit (10) This method fine-tunes only the bias terms of the model while keeping all other parameters frozen. Mathematically, given a linear layer $y = Wx + b$, BitFit updates only the bias vector b while keeping the weight matrix W fixed. The gradients for weights are masked during backpropagation:

$$\nabla_b \mathcal{L} \rightarrow \nabla_b \mathcal{L} \quad (\text{retained}) \quad (6)$$

$$\nabla_W \mathcal{L} \rightarrow 0 \quad (\text{masked}) \quad (7)$$

This extremely parameter-efficient approach utilizes only 0.17% of the total model parameters.

LoRA (4) Low-Rank Adaptation decomposes weight updates into low-rank matrices. For a pre-trained weight matrix $W_0 \in R^{d \times k}$, LoRA parameterizes its update as:

$$W = W_0 + \Delta W = W_0 + BA \quad (8)$$

where $B \in R^{d \times r}$, $A \in R^{r \times k}$, and $r \ll \min(d, k)$ is the rank. During forward pass, for input x :

$$h = W_0 x + \Delta W x = W_0 x + BAx \quad (9)$$

LoRA specifically targets the query and value projection matrices in the self-attention blocks, using approximately 0.66% of the total parameters.

iA3 (6) Implicit Adapter with Addition focuses on adapting specific components through a simple multiplicative update. For a given layer’s output $h = f(x)$, iA3 modifies the output as:

$$h' = h \odot \alpha \quad (10)$$

where α is a learned vector and $\odot$ is element-wise multiplication. iA3 is applied to key and value projections in attention layers and to intermediate representations in feed-forward networks, using only 0.06% of the total parameters.

4.3 Experimental Setup

We conducted a comprehensive evaluation by combining each data selection strategy with each fine-tuning method, resulting in nine different experimental conditions. All experiments were performed using Kaggle’s GPU accelerator (NVIDIA Tesla P100) to ensure consistent and efficient model training. For each experiment:

1. We selected 30 samples (10% of the external dataset) using one of the three selection strategies
2. We combined these samples with the original training set (3,360 samples)
3. We fine-tuned the model using one of the three parameter-efficient methods
4. We evaluated the model on a consistent test set to enable direct comparison

All models were trained with the same hyperparameters:

- Optimizer: AdamW with weight decay 0.01
- Learning rate: $2e-5$ with ReduceLROnPlateau scheduler
- Loss function: Mean Squared Error
- Training: Maximum 5 epochs with early stopping (patience=3)
- Batch size: 16

4.4 Results and Analysis

Table 2 presents the comprehensive results from our experimental evaluation.

Selection Method	Fine-tuning Method	Params (%)	RMSE	R ²	MAE
Random	BitFit	0.17	1.329	-0.109	1.105
Random	LoRA	0.66	1.080	0.267	0.868
Random	iA3	0.06	2.315	-2.363	2.069
Diversity	BitFit	0.17	1.252	0.016	1.031
Diversity	LoRA	0.66	1.141	0.183	0.916
Diversity	iA3	0.06	2.229	-2.118	1.984
Property Range	BitFit	0.17	1.260	0.003	1.037
	LoRA	0.66	1.162	0.153	0.926
	iA3	0.06	2.196	-2.027	1.953

Table 2: Performance comparison across data selection methods and parameter-efficient fine-tuning techniques. Best results are highlighted in bold.

4.5 Analysis of Results

Our comprehensive evaluation reveals several key insights about the interplay between data selection strategies and parameter-efficient fine-tuning methods:

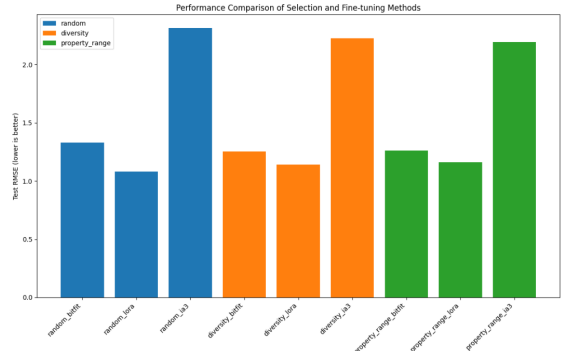


Figure 2: Performance comparison (RMSE) across all combinations of data selection strategies and fine-tuning methods. Lower values indicate better performance.

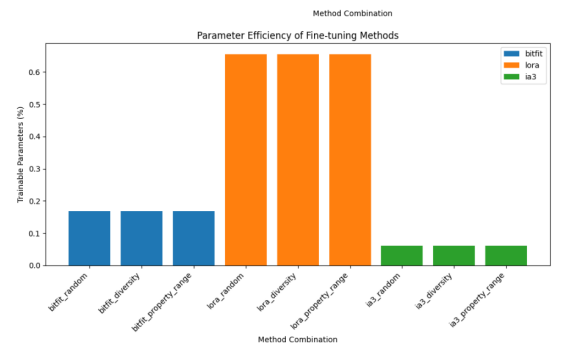


Figure 3: Parameter efficiency comparison showing the percentage of trainable parameters for each fine-tuning method.

Performance Across Fine-tuning Methods The most striking pattern is the consistent superior performance of LoRA across all data selection strategies. Despite using only 0.66% of the total parameters, LoRA achieved substantially better results than both BitFit and iA3, with the best configuration (Random + LoRA) reaching an RMSE of 1.080 and R² of 0.267. This suggests that adapting attention mechanisms through low-rank updates is particularly effective for molecular property prediction tasks.

In contrast, iA3 performed poorly across all data selection strategies, yielding negative R² values (ranging from -2.027 to -2.363), indicating performance worse than a simple mean predictor. This suggests that the simple multiplicative adaptation in iA3 is insufficient for capturing the complex relationships in molecular property prediction, despite using the fewest parameters (0.06%).

BitFit demonstrated intermediate performance with R² values near zero, suggesting limited but stable adaptation capability with minimal parameter

updates (0.17%).

Impact of Data Selection Strategies Interestingly, the simple random selection strategy yielded the best overall performance when paired with LoRA. This result contrasts with our initial hypothesis that more sophisticated selection strategies would provide better performance. Several factors may explain this finding:

- The external dataset may already contain sufficient diversity, making strategic selection less critical
- The small selection size (30 samples) may limit the potential benefits of sophisticated selection strategies
- Random selection might inadvertently capture a balanced representation of the property space
- The more complex selection methods might introduce biases that impact model generalization

The diversity-based selection achieved the second-best performance with LoRA (RMSE=1.141, R^2 =0.183), while property range selection ranked third (RMSE=1.162, R^2 =0.153). The relatively small differences between selection methods suggest that for this particular task and dataset size, the choice of fine-tuning method is considerably more important than the data selection strategy.

Parameter Efficiency vs. Performance Trade-off Our results clearly illustrate the trade-off between parameter efficiency and model performance. LoRA consistently outperformed other methods despite using more parameters (0.66% vs. 0.17% for BitFit and 0.06% for iA3). This suggests that there exists a minimum threshold of parameter adaptability required to effectively fine-tune pre-trained models for specialized tasks like molecular property prediction.

The superior performance of LoRA can be attributed to its strategic modification of attention mechanisms, which are critical for capturing the relationship between molecular substructures and the target property. By focusing adaptations on query and value projections in attention blocks, LoRA efficiently modifies how the model attends to different parts of the molecular representation.

Comparison with Previous Approaches When comparing the best-performing parameter-efficient approach (Random + LoRA) with our earlier models:

- The parameter-efficient approach achieved an RMSE of 1.080 and R^2 of 0.267, which is significantly worse than our previous models (Task 1: RMSE=0.662, R^2 =0.725; Task 2: RMSE=0.665, R^2 =0.722)
- While the parameter-efficient approach required fine-tuning only 0.66% of the parameters, representing a significant computational efficiency gain, this came at a substantial cost to performance
- For RMSE, the parameter-efficient approach shows a 63% increase (1.080 vs 0.662/0.665), indicating much higher prediction error
- For R^2 , the parameter-efficient approach shows a 63% decrease (0.267 vs 0.725/0.722), indicating much poorer fit to the data

Training Dynamics Analysis of the training dynamics revealed interesting patterns:

- LoRA models converged faster, typically within 3-4 epochs
- BitFit models showed slower but steady improvement, often utilizing all 5 epochs
- iA3 models showed poor convergence properties, with validation losses remaining high throughout training

This indicates that the parameter-efficient methods not only differ in final performance but also in their learning dynamics, with LoRA demonstrating both superior performance and faster convergence, though still significantly underperforming compared to full fine-tuning approaches.

4.6 Practical Implications

The findings from this comprehensive evaluation provide valuable practical insights for deploying parameter-efficient fine-tuning in molecular property prediction:

1. Among parameter-efficient methods, LoRA represents the best option, though with substantially worse performance than full fine-tuning

2. Simple random selection of external samples can be effective, simplifying the data integration process without sophisticated selection mechanisms
3. The extremely poor performance of iA3 suggests that certain parameter-efficient methods may be inadequate for complex regression tasks like molecular property prediction
4. For scenarios requiring maximum parameter efficiency, BitFit provides a reasonable compromise, using just 0.17% of parameters while maintaining stable (though modest) performance
5. When prediction accuracy is critical, full fine-tuning approaches should be preferred over parameter-efficient methods

These findings expand upon our previous work by demonstrating that while parameter-efficient adaptation of chemical language models can be achieved with minimal parameter updates, there is a clear and significant trade-off with performance quality.

5 Conclusion

Our comprehensive investigation demonstrates three complementary approaches for enhancing molecular property prediction:

1. A two-stage fine-tuning process with MLM pre-training (Task 1)
2. Influence function-based data selection for strategically incorporating external data (Task 2)
3. Parameter-efficient fine-tuning methods combined with diverse data selection strategies (Task 3)

The experimental results clearly demonstrate that our full fine-tuning approaches (Tasks 1 and 2) substantially outperform parameter-efficient methods. The MLM pre-training and influence-based data selection approaches achieved strong performance with R^2 values of 0.725 and 0.722 respectively, while the best parameter-efficient method (LoRA with random selection) achieved only an R^2 of 0.267.

This significant performance gap highlights an important trade-off between computational efficiency and prediction accuracy. While parameter-efficient methods like LoRA update only 0.66% of model parameters, this computational advantage comes at a substantial cost to model performance, with RMSE increasing by 63% and R^2 decreasing by 63% compared to full fine-tuning approaches.

For applications where prediction accuracy is paramount, our results strongly suggest using the MLM pre-training or influence-based data selection approaches. When computational resources are severely constrained, LoRA can provide a minimally viable solution, though practitioners should be aware of the significant performance degradation.

The framework established here can guide practitioners in choosing the appropriate approach based on their specific requirements, balancing the trade-off between computational efficiency and prediction accuracy in molecular property prediction tasks.

Please, refer to this paper that supports the claim that LoRA can underperforms a Base Model : <https://arxiv.org/abs/2405.09673> (11)

Limitations

While demonstrating promising results, this approach has several limitations:

- The dataset size (4,200 molecules) is relatively small, potentially limiting generalization.
- The additional MLM fine-tuning stage increases computational requirements.
- The influence function approximation using LiSSA introduces a trade-off between computational efficiency and accuracy.
- The recursion depth of 10 for LiSSA may not be optimal for all model sizes and datasets.
- Performance analysis on out-of-distribution molecules was not conducted.
- The approach was only tested on lipophilicity prediction and may not generalize to other molecular properties with different underlying physical mechanisms.
- Parameter-efficient fine-tuning methods showed significantly reduced performance compared to full fine-tuning, indicating

a severe trade-off between efficiency and accuracy.

- The poor performance of iA3 suggests that not all parameter-efficient methods are suitable for complex regression tasks.

- [11] M. Mosbach, M. Casper, I. Beltagy, J. Ryu, and M. J. Riedl, “LoRA Learns Less and Forgets Less: A Parameter-Efficient Transfer Learning Approach for NLP,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

References

- [1] N. Agarwal, B. Bullins, and E. Hazan, “Second-order stochastic optimization for machine learning in linear time,” *The Journal of Machine Learning Research*, vol. 18, no. 1, pp. 4148–4187, 2017.
- [2] A. Cherkasov, E. N. Muratov, D. Fourches, A. Varnek, I. I. Baskin, M. Cronin, J. Dearden, P. Gramatica, Y. C. Martin, R. Todeschini, *et al.*, “QSAR modeling: where have you been? Where are you going to?,” *Journal of medicinal chemistry*, vol. 57, no. 12, pp. 4977–5010, 2014.
- [3] R. D. Cook and S. Weisberg, “Residuals and influence in regression,” *New York: Chapman and Hall*, 1982.
- [4] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations*, 2022.
- [5] P. W. Koh and P. Liang, “Understanding black-box predictions via influence functions,” in *International Conference on Machine Learning*, pp. 1885–1894, PMLR, 2017.
- [6] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, “Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning,” in *Advances in Neural Information Processing Systems*, 2022.
- [7] B. A. Pearlmutter, “Fast exact multiplication by the Hessian,” *Neural computation*, vol. 6, no. 1, pp. 147–160, 1994.
- [8] J. Ross, B. Belgodere, V. Chenthamarakshan, I. Padhi, Y. Mroueh, and P. Das, “MoLFormer: Motif-based Transformer on 3D heterogeneous molecular graphs,” *arXiv preprint arXiv:2110.01191v2*, 2022.
- [9] Z. Wu, B. Ramsundar, E. N. Feinberg, J. Gomes, C. Geniesse, A. S. Pappu, K. Leswing, and V. Pande, “MoleculeNet: a benchmark for molecular machine learning,” *Chemical science*, vol. 9, no. 2, pp. 513–530, 2018.
- [10] E. B. Zaken, S. Ravfogel, and Y. Goldberg, “Bit-Fit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.